

Print Job Interceptor - Coding Challenge

Overview

Create a robust Windows desktop application that can intercept, monitor, and manage print jobs in real-time. This challenge tests your ability to work with Windows APIs, handle asynchronous operations, implement clean architecture, and solve complex logical problems.

Core Requirements

1. Print Job Monitoring

- **Objective:** Monitor the Windows print spooler for new print jobs
- **Implementation:** Use Windows Print Spooler API or WMI events to detect new print jobs
- **Key Points:** Handle multiple printers, manage event subscriptions properly

2. Print Job Management

- **Pause/Resume Functionality:** Ability to pause and resume individual print jobs
- **Job Information Extraction:** Capture job details (document name, user, printer, page count, etc.)
- **Status Tracking:** Monitor job status changes (printing, paused, completed, error)

3. Document Grouping Logic

- **Challenge:** Some documents generate multiple print jobs (e.g., complex PDFs, multi-part documents)
- **Solution:** Implement intelligent grouping based on:
 - Time proximity (configurable timeout, suggest 5-10 seconds)
 - User and document name correlation
 - Job sequence patterns
- **Edge Cases:** Handle rapid successive jobs, identical document names, different users

4. Print Job Analysis

- **Basic Parsing:** Extract document metadata where possible
- **Content Analysis:** Identify document type (text, image, mixed)

Technical Requirements

Architecture

- **Pattern:** MVVM (WPF) or MVP (WinForms)
- **Separation of Concerns:** Clear separation between UI, business logic, and data access
- **Dependency Injection:** Use built-in .NET DI container or similar

Error Handling & Robustness

- **Service Recovery:** Handle print spooler service restarts
- **Exception Management:** Graceful handling of API errors
- **Logging:** Comprehensive logging for debugging and monitoring
- **Memory Management:** Proper disposal of resources and event handlers

Performance Considerations

- **Async Operations:** Non-blocking UI during print job operations
- **Threading:** Safe cross-thread operations for UI updates
- **Resource Management:** Efficient handling of Windows API calls

Deliverables

1. Desktop Application

Core Features:

- Real-time print job list with status updates
- Pause/Resume controls for individual jobs
- Document grouping visualization
- Basic job details display
- Simple configuration panel (timeout settings, enabled printers)

UI Requirements:

- Clean, professional interface
- Responsive design (no freezing during operations)
- Clear status indicators
- Error messaging

2. Code Quality

- **Documentation:** Clear comments for complex logic
- **Testing:** Unit tests for core business logic (bonus points)
- **Clean Code:** Readable, maintainable code structure

3. Technical Documentation

Brief written explanation of:

- Architecture decisions and trade-offs
- Print job grouping algorithm logic
- Known limitations and potential improvements
- Deployment considerations for enterprise environments

Submission Guidelines

- **Format:** ZIP file or Git repository
- **Include:** Source code, compiled executable, brief documentation
- **Bonus:** Include test plan or demo scenarios

API Hints & Resources

- **Windows Print Spooler API:** `winspool.drv` functions
- **WMI Events:** `Win32_PrintJob` class for monitoring
- **Consider:** Print job states, spooler file locations, security contexts

Stretch Goals (Optional)

- **Advanced Parsing:** Extract text content from simple print jobs
 - **Configuration Persistence:** Save/load application settings
 - **Print Preview:** Basic preview of intercepted documents
 - **Multiple Printer Support:** Handle different printer types elegantly
 - **Installation Package:** Create simple MSI or setup executable
-